



GROUPE 3

CLOUD NATIVE & CONTENEURISATION

Virtualisation et Ing des
systèmes cloud devops



MEMBRES ÉQUIPE

Abdou **SIDIBÉ**

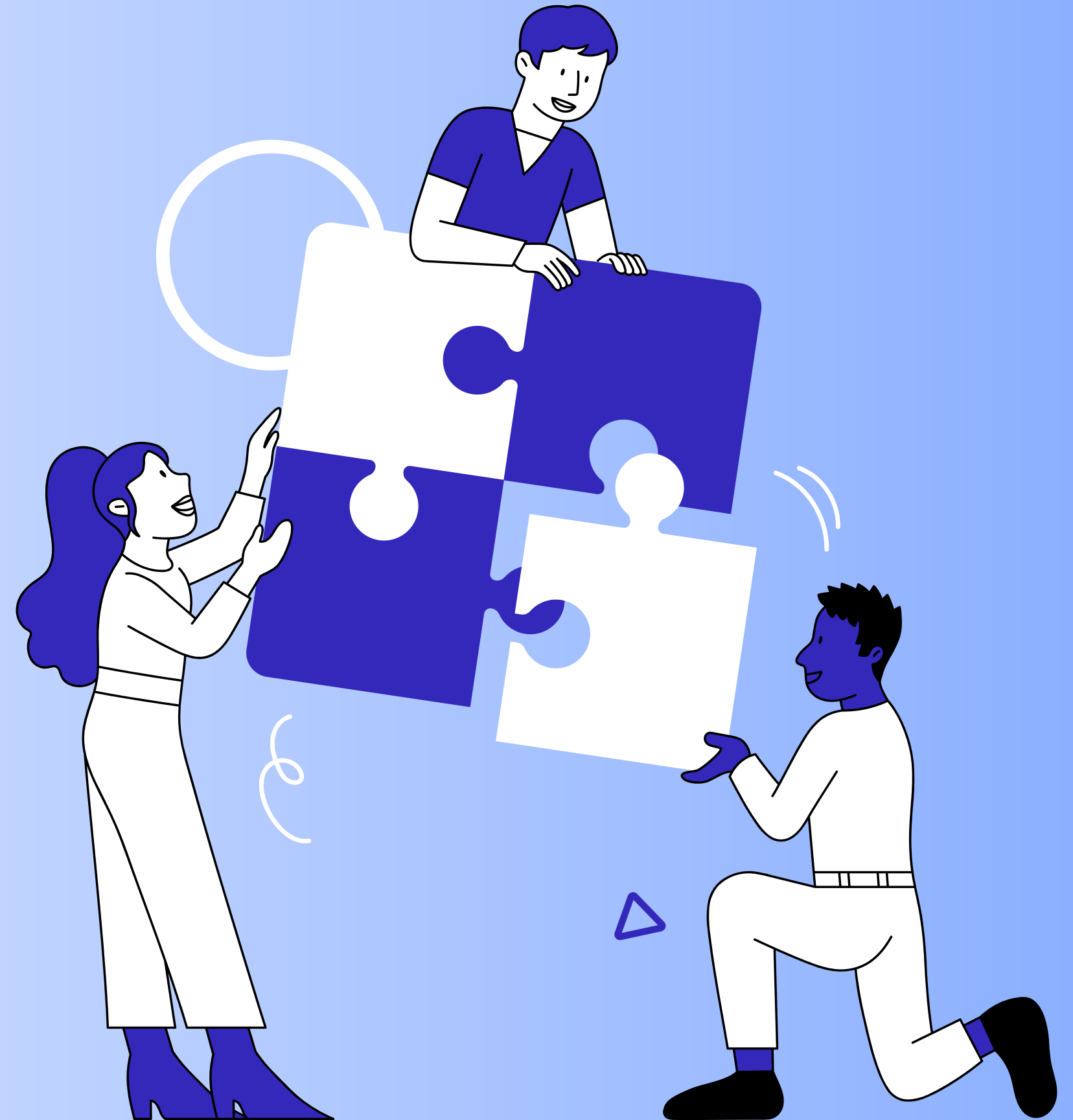
Yaya **Bagayoko**

Ahmed Noumory **Diakité**

Etienne **KAMATÉ**

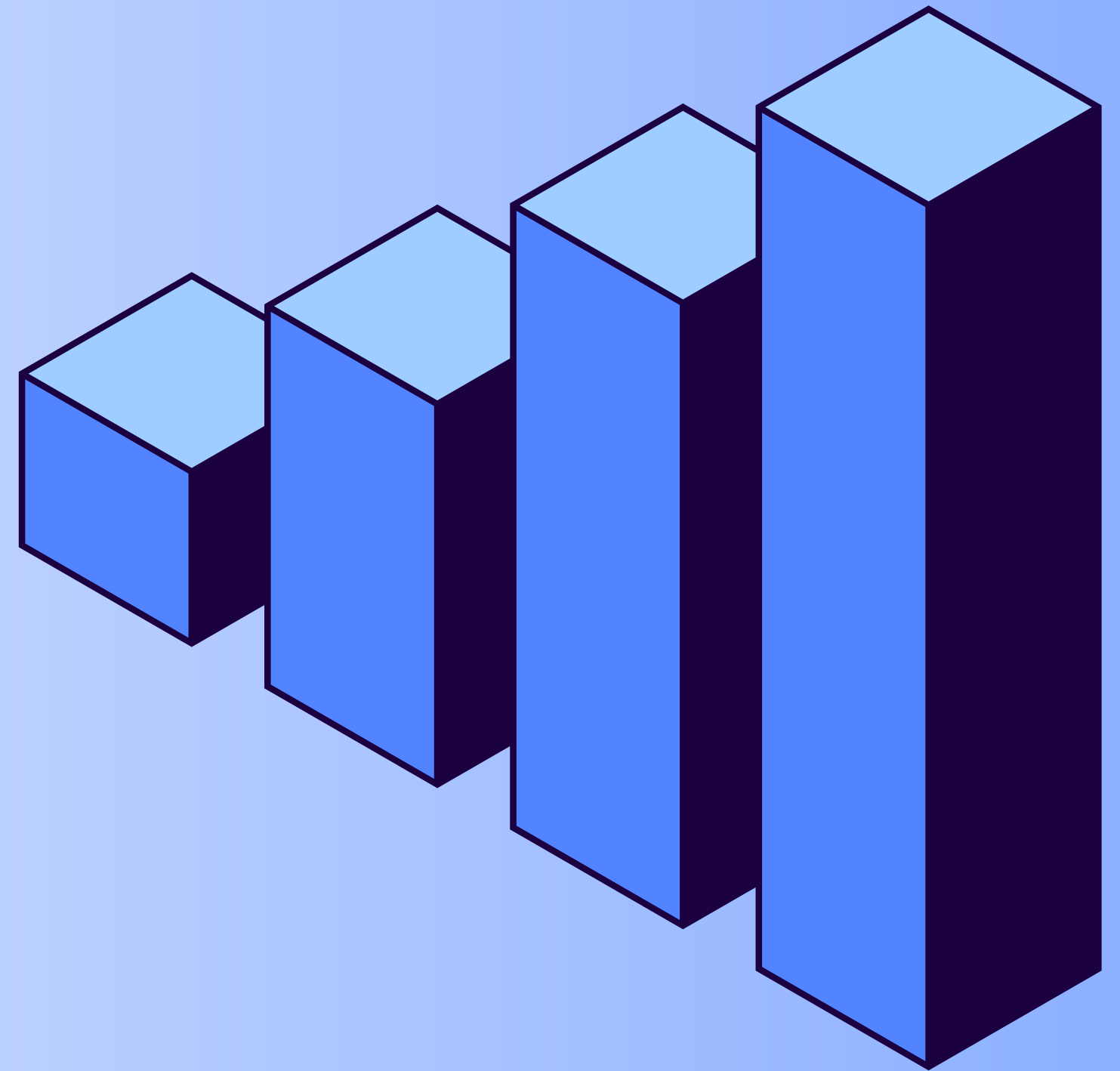
Ibrahim **Touré**

Mady **Dembélé**

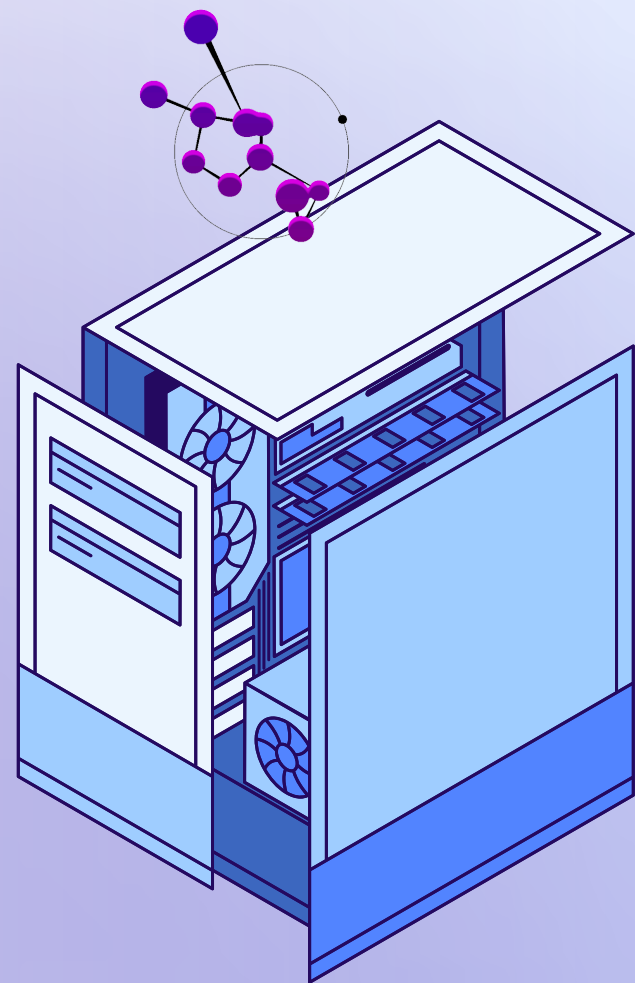


Introduction

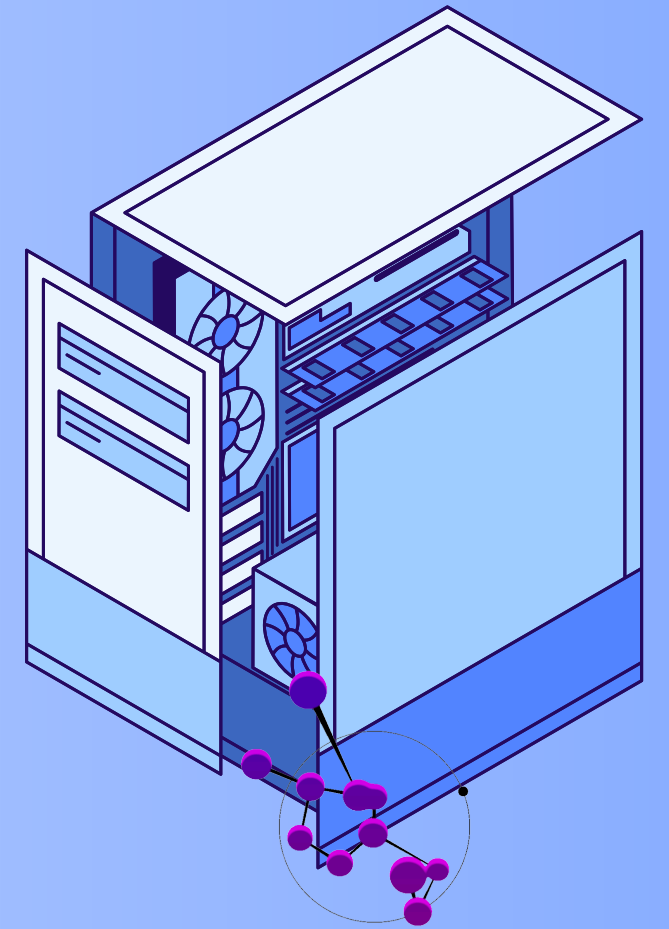
MADY DEMBÉLÉ



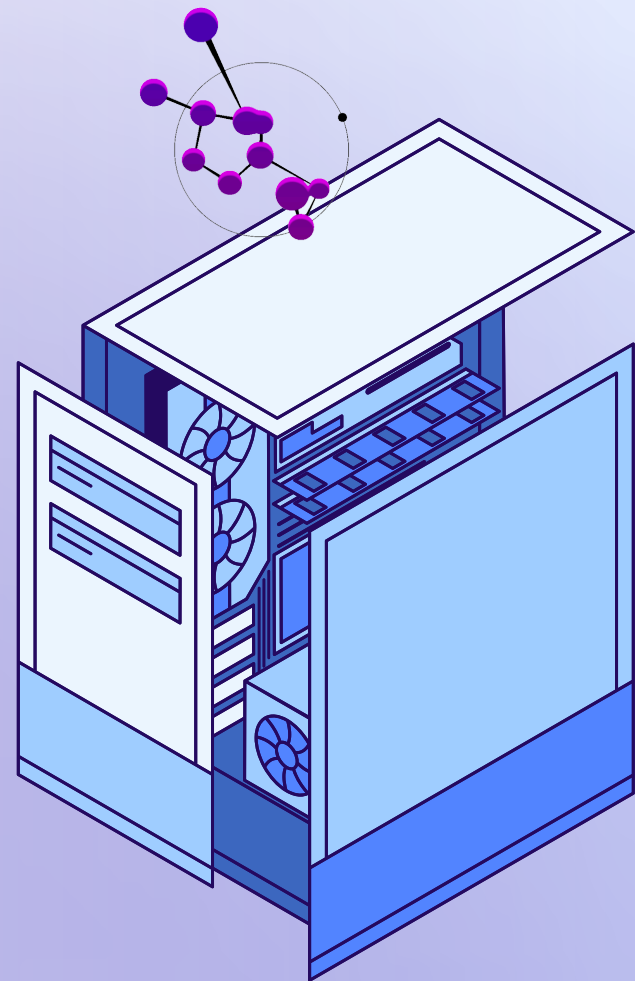
QU'EST-CE QUE L'ARCHITECTURE CLOUD-NATIVE ?



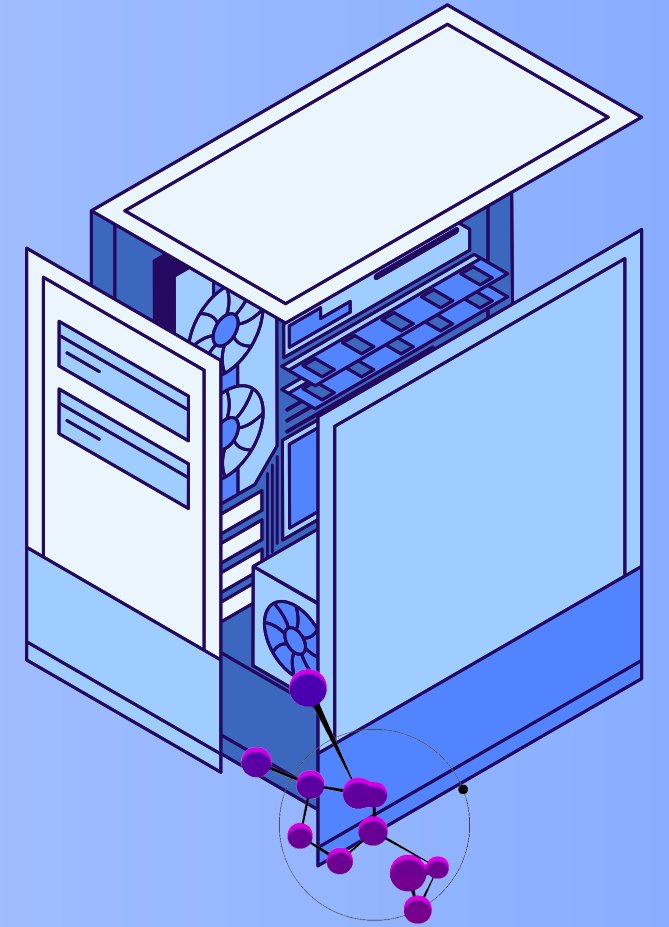
- Automatisation et CI/CD : Implémentation de pipelines automatisés de livraison continue pour garantir des déploiements fiables et reproductibles.
- Microservices : Découpage des applications en services indépendants (ex: frontend, catalogue, moteur de recommandation).
- Infrastructure as Code (IaC) : Formalisation de l'infrastructure via du code pour éviter les erreurs humaines et la configuration manuelle.
- Résilience : Tolérance aux pannes, auto-scaling et haute disponibilité au cœur du système.



QU'EST-CE QUE LA CONTENEURISATION ?



La conteneurisation est une méthode de développement logiciel qui consiste à encapsuler une application et toutes ses dépendances (fichiers, bibliothèques, configurations) dans une unité autonome appelée **conteneur**. Cela garantit que le programme fonctionne de manière fluide et identique sur n'importe quelle infrastructure, qu'il s'agisse d'un ordinateur portable ou d'un serveur dans le cloud.

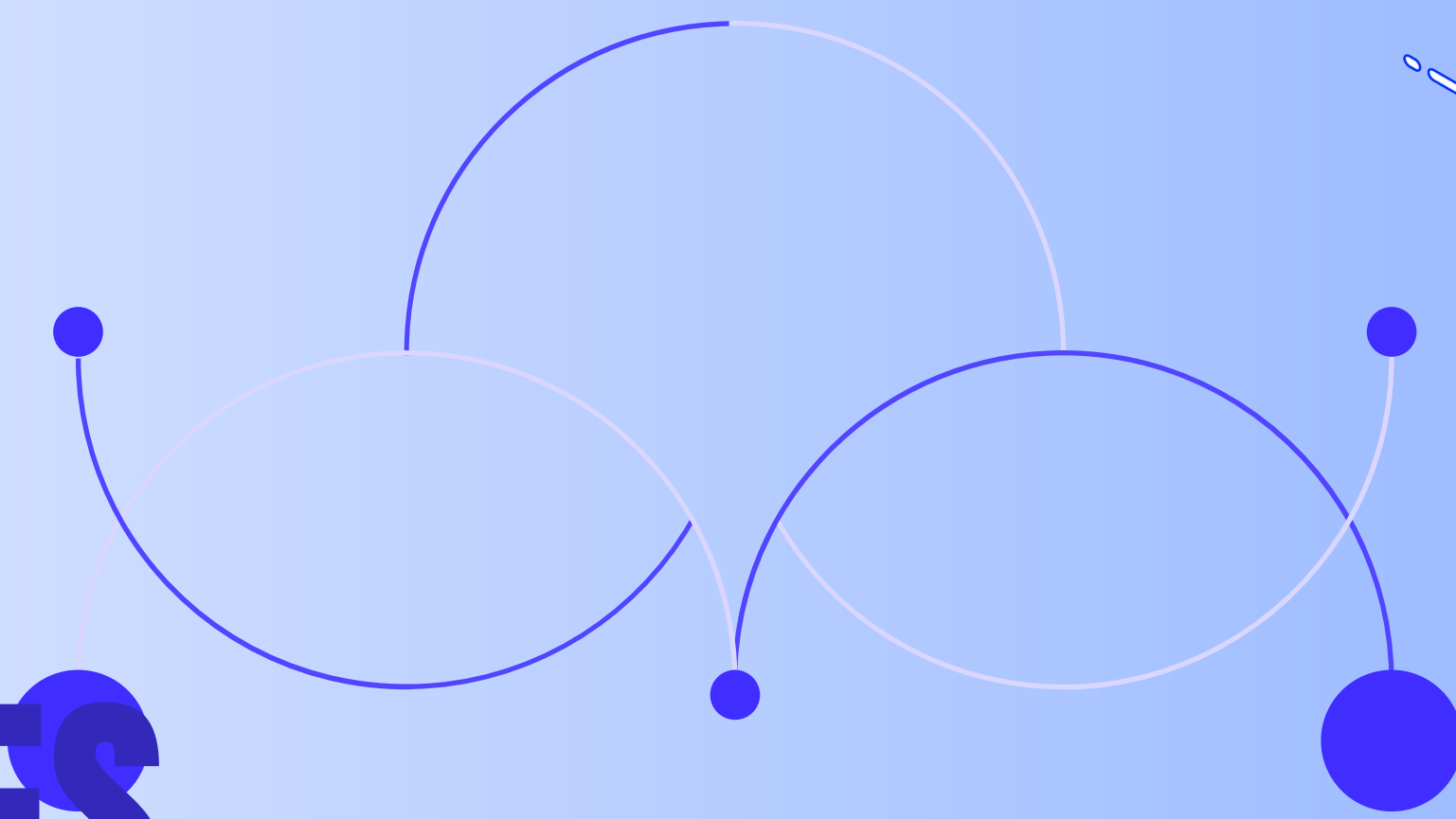


Kubernetes

YAYA BAGAYOKO



KUBERNETES

An abstract graphic consisting of several blue circles of varying sizes and thin blue arcs connecting them, set against a light blue background. The circles are positioned at the top and bottom of the frame, with arcs curving between them.

L'CHEF D'ORCHESTRE

Kubernetes est une plateforme open source d'orchestration permettant de gérer des milliers de conteneurs de manière automatisée. Créé initialement par Google puis maintenu par la Cloud Native Computing Foundation.



POURQUOI KUBERNETES ?



Problèmes rencontrés avec Docker seul

- Gestion de plusieurs serveurs
- Scalabilité manuelle
- Tolérance aux pannes limitée
- Déploiements complexes

Solutions apportées

- Auto-scaling
- Self-healing
- Rolling updates
- Load balancing
- Gestion des secrets



ARCHITECTURE KUBERNETES



Control Plane

- API Server
- Scheduler
- Controller Manager
- etcd

Worker Nodes

- Kubelet
- Container Runtime
- Pods



CONCEPTS FONDAMENTAUX KUBERNETES



- Pod : Plus petite unité déployable.
- Deployment : Gestion du cycle de vie des Pods.
- Namespace : Permet l'isolation logique des ressources.
- Auto-scaling : Horizontal Pod Autoscaler (HPA), permet d'ajuster automatiquement le nombre de Pods.

Service : Expose les Pods.

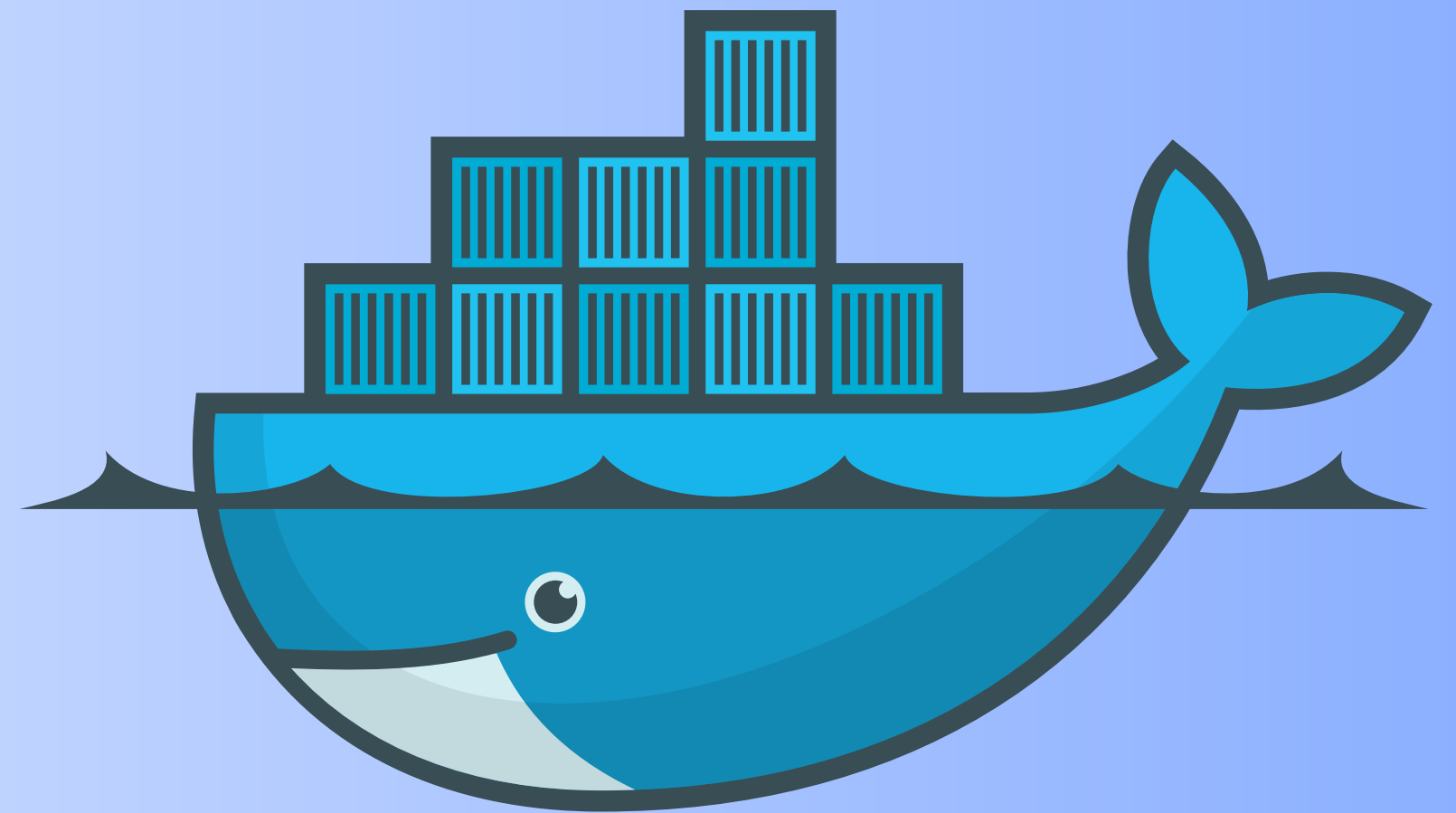
Types :

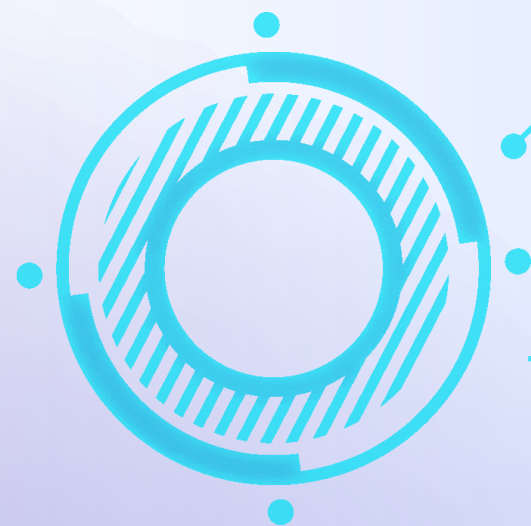
- ClusterIP
- NodePort
- LoadBalancer
- ExternalName



Docker

ETIENNE KAMATÉ





DOCKER

Définition

Docker est une plateforme permettant d'empaqueter une application et ses dépendances dans des conteneurs légers et portables.

Pourquoi Docker ?

Avant Docker :

- Problème du "Ça marche sur ma machine"
- Dépendances incompatibles
- Déploiements complexes

Avec Docker :

- Environnement standardisé
- Portabilité
- Isolation
- Rapidité de déploiement

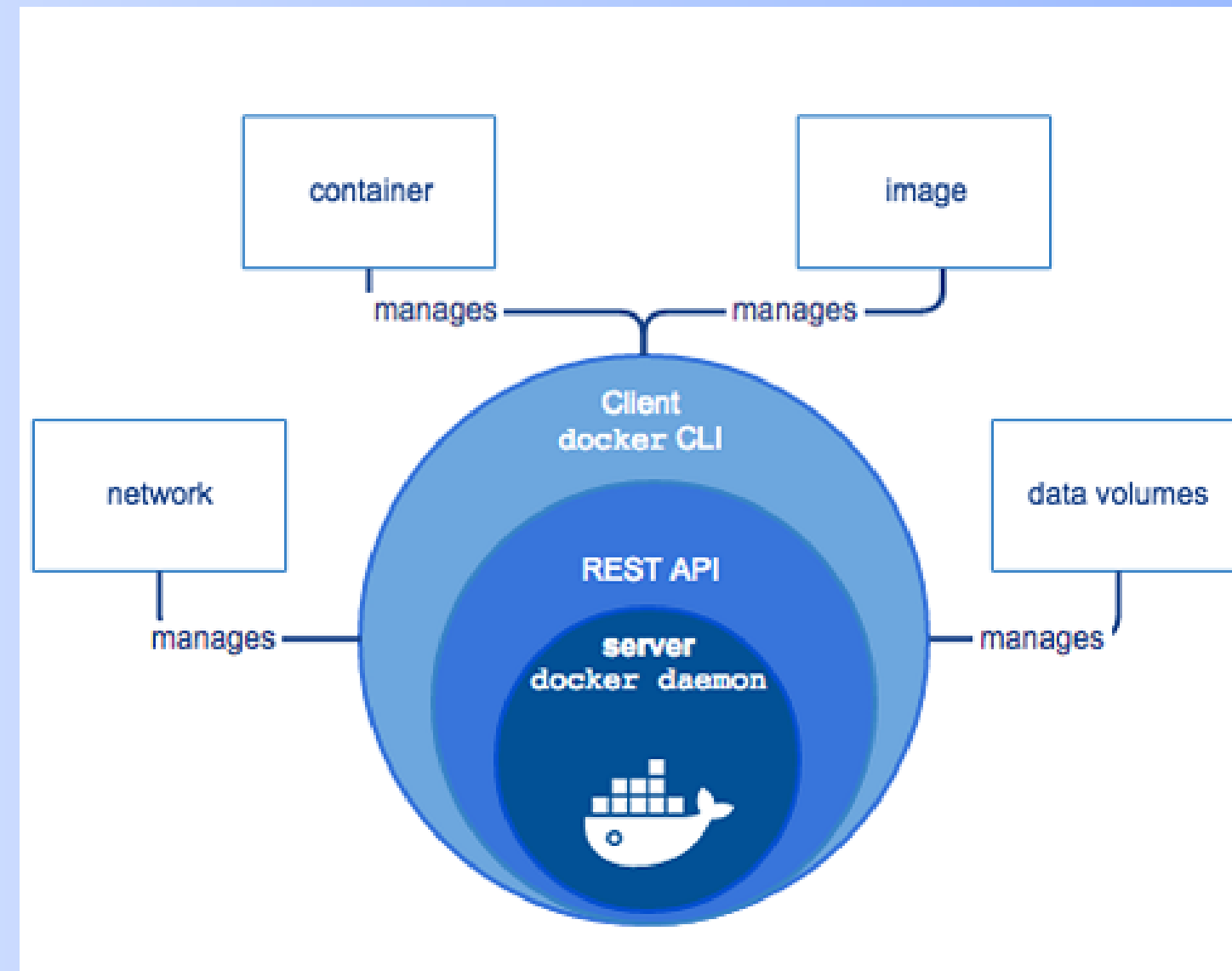


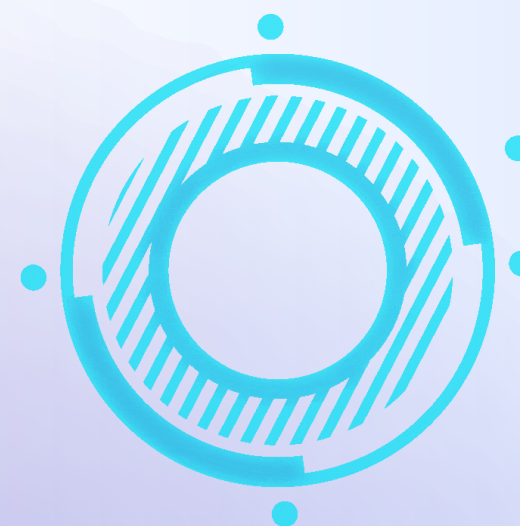
ARCHITECTURE DOCKER

- **Docker Client**

- **Docker Daemon**

- **Containers**
- **Images**
- **Networks**
- **Volumes**





CONCEPTS CLÉS

Image : Template immuable servant à créer un conteneur.

Exemples :

- nginx
- ubuntu
- node
- postgres

Conteneur : Instance en exécution d'une image.

Registry : Stockage des images.



EXEMPLE DOCKERFILE

FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY ..

EXPOSE 3000

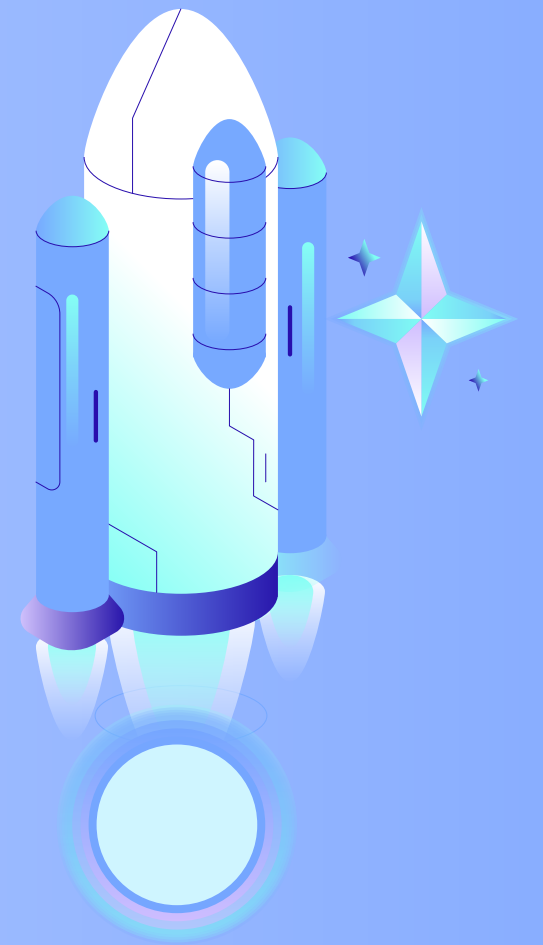
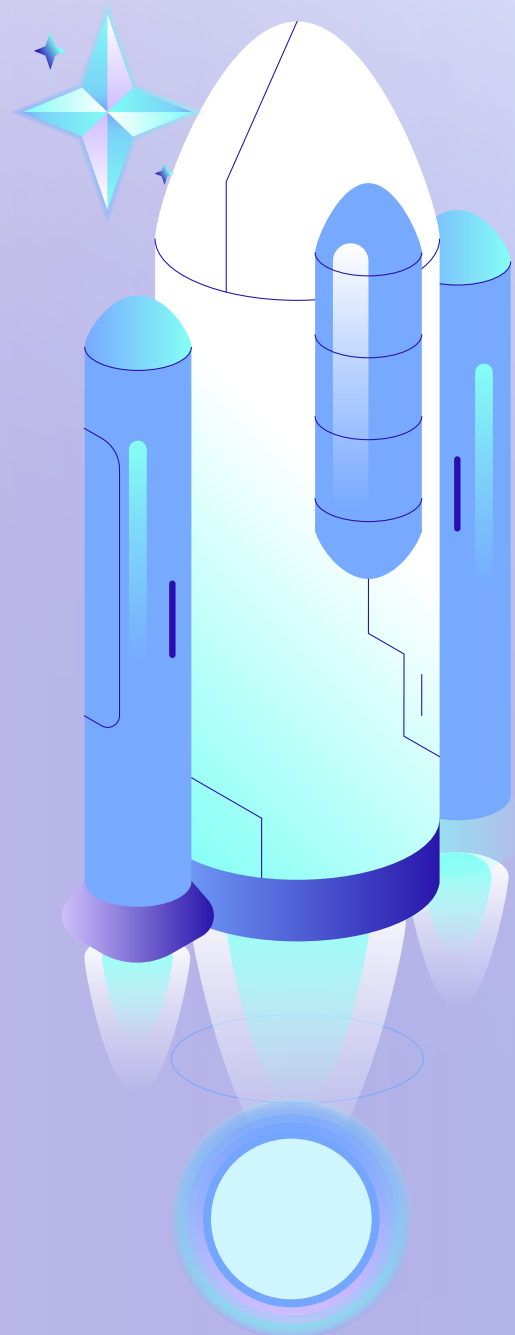
CMD ["npm", "start"]

- Construction :

docker build -t myapp .

- Exécution :

docker run -p 3000:3000 myapp



Ingress Controller

AHMED DIAKITÉ



QU'EST-CE QU'UN INGRESS ?

Un Ingress est une ressource Kubernetes permettant de gérer l'accès HTTP/HTTPS aux services internes du cluster.



Sans Ingress :

- Internet
|
- Service A (LB)
- Service B (LB)
- Service C (LB)

Avec Ingress :

- Internet
|
- Ingress Controller
- |—— app.company.com
- |—— api.company.com
- |—— admin.company.com



INGRESS CONTROLLER

Définition : L'Ingress Controller implémente les règles définies dans les objets Ingress.

Solutions populaires :

- NGINX Ingress Controller
- Traefik
- HAProxy Ingress
- Kong



EXEMPLE D'INGRESS

Fonctionnalités :

- Routage HTTP
- SSL/TLS
- Load Balancing
- Réécriture d'URL
- Authentification

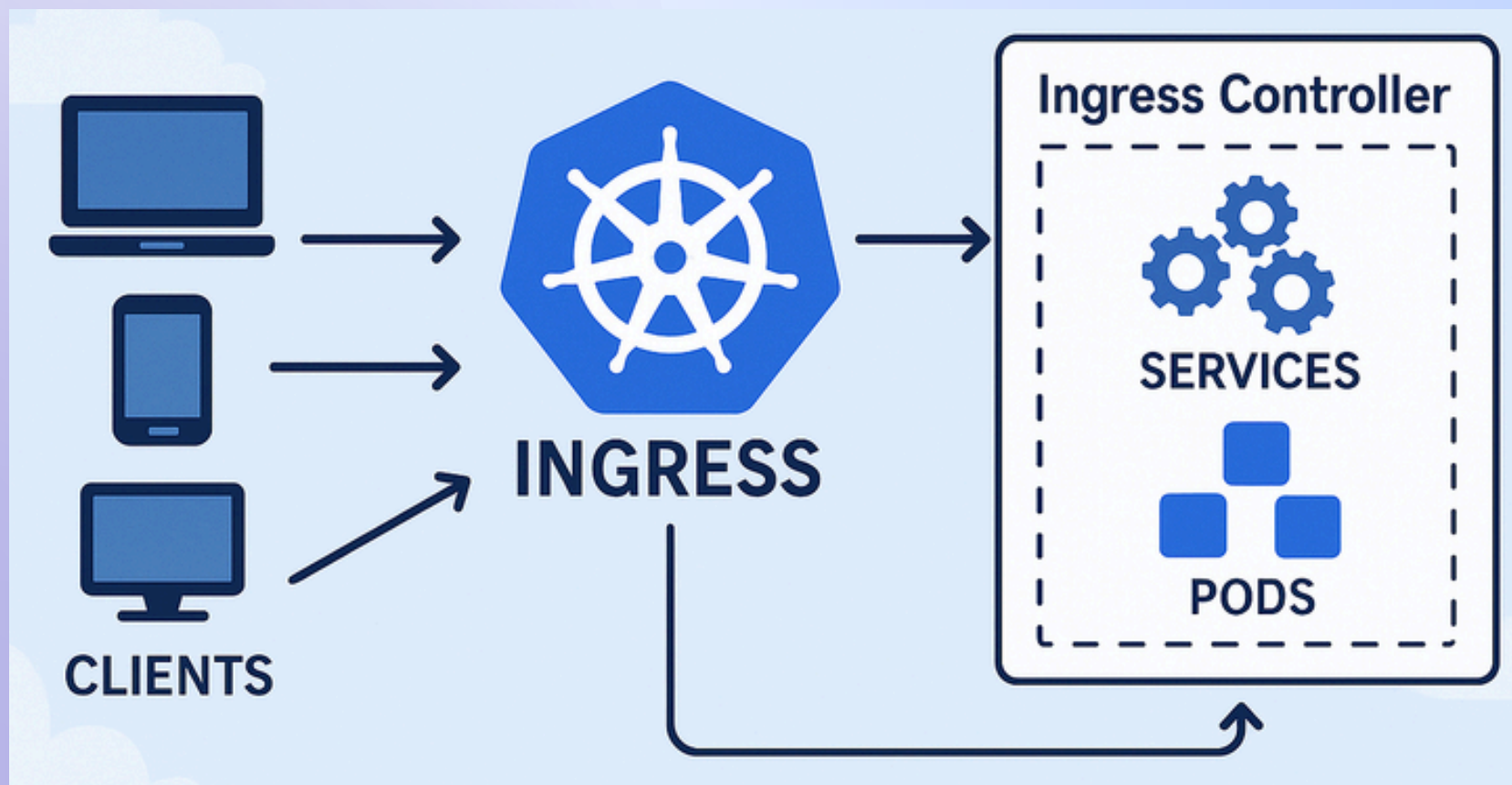


```
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```
metadata:
  name: app-ingress
```

```
spec:
  ingressClassName: nginx
```

```
rules:
- host: app.company.com
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: web-service
          port:
            number: 80
```



EXEMPLE D'INGRESS

Définition : L'Ingress Controller implémente les règles définies dans les objets Ingress.

Solutions populaires :

- NGINX Ingress Controller
- Traefik
- HAProxy Ingress
- Kong



Helm

IBRAHIM TOURÉ





INTRODUCTION À HELM

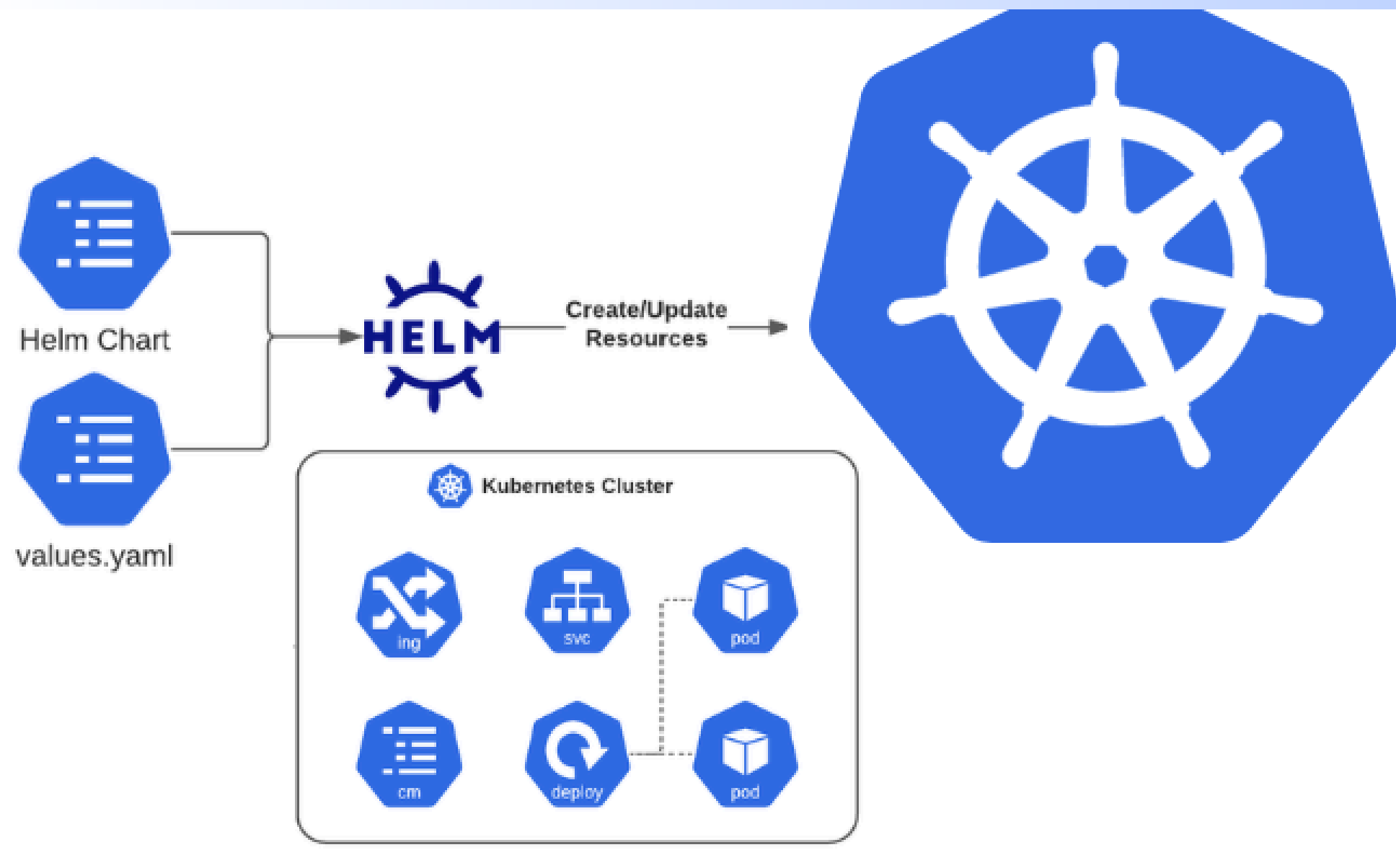
Définition : Helm est le gestionnaire de packages de Kubernetes.

Comparable à :

- apt pour Linux
- yum
- npm
- pip



POURQUOI HELM ?



Problème : Déploiement manuel

- deployment.yaml
- service.yaml
- ingress.yaml
- configmap.yaml
- secret.yaml

Complexité :

- Gestion des versions
- Répétition
- Maintenance difficile

Solution : Helm permet

- Templates
- Paramétrage
- Versionning
- Réutilisation



STRUCTURE D'UN CHART HELM

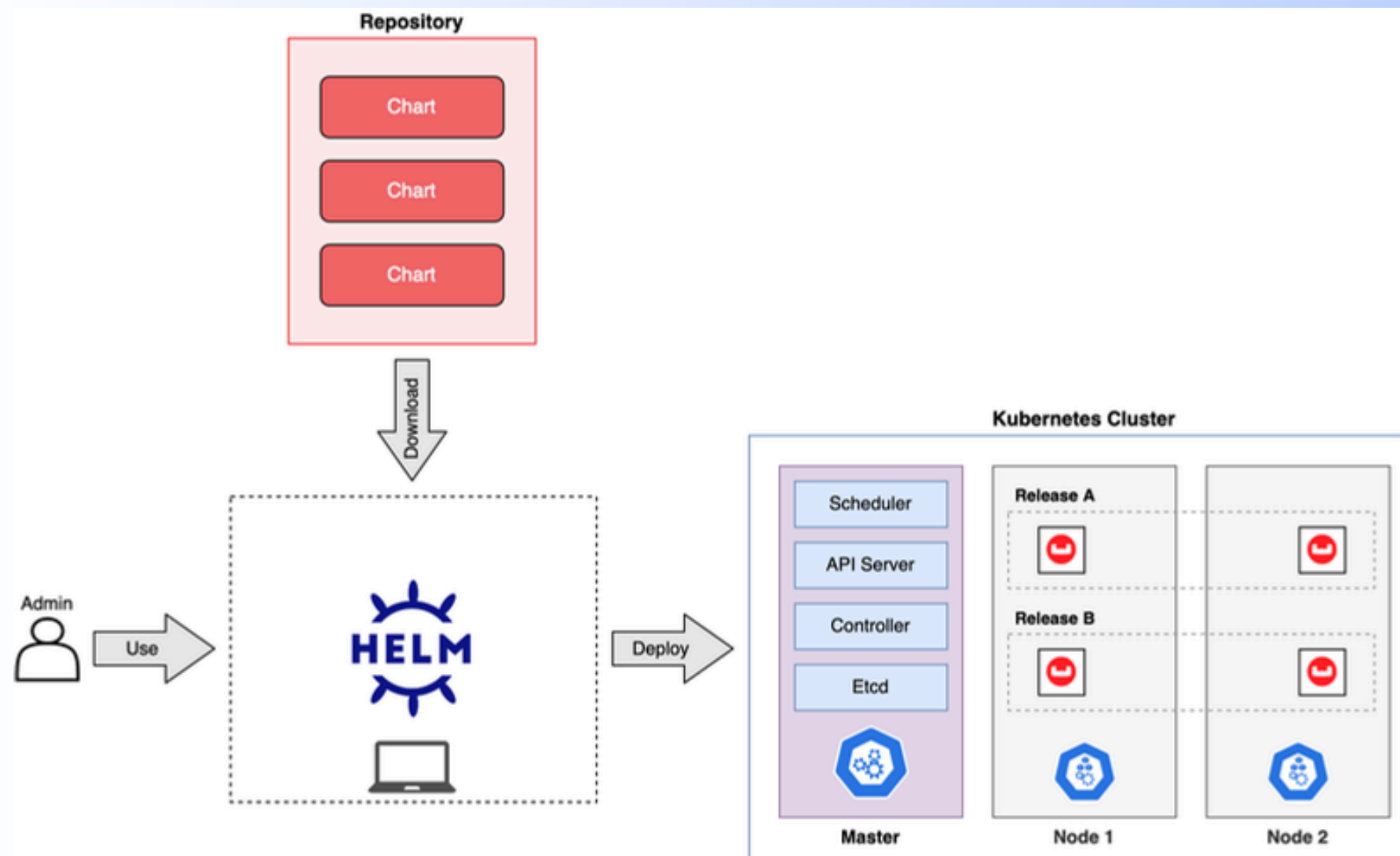


```
mychart/  
├── Chart.yaml  
├── values.yaml  
├── templates/  
│   ├── deployment.yaml  
│   ├── service.yaml  
│   └── ingress.yaml  
└── charts/
```





COMMANDES HELM ESSENTIELLES



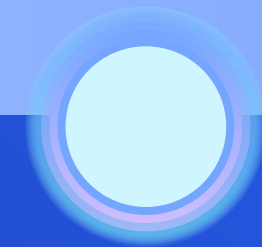
- Installation : `helm install myapp ./mychart`
- Mise à jour : `helm upgrade myapp ./mychart`
- Rollback : `helm rollback myapp 1`
- Suppression : `helm uninstall myapp`





CAS PRATIQUE COMPLET

CYCLE DE DÉPLOIEMENT CLOUD NATIVE



Developer



Git Push



CI/CD Pipeline



Docker Build



Container Registry



Helm Deploy



Kubernetes Cluster



Ingress Controller

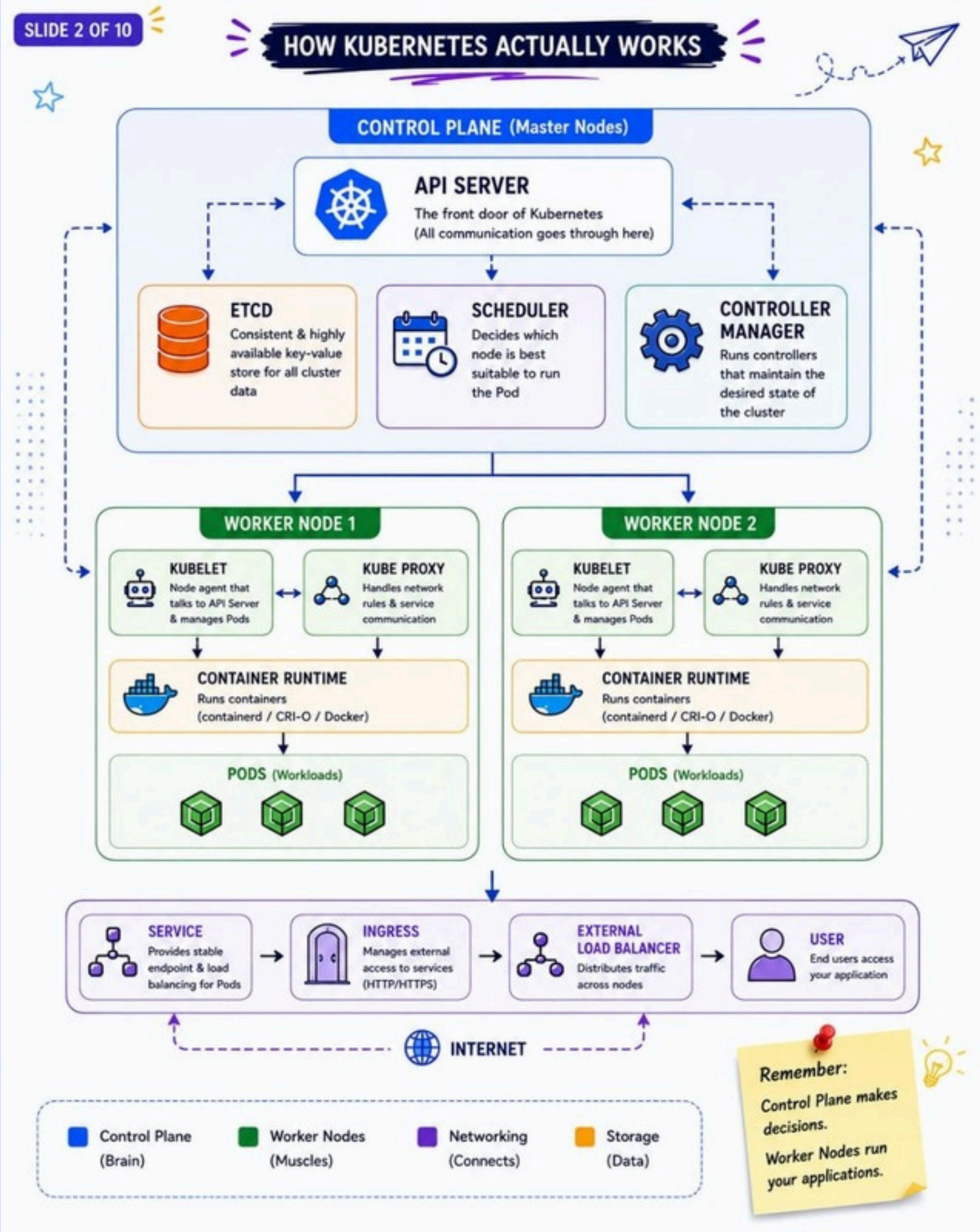
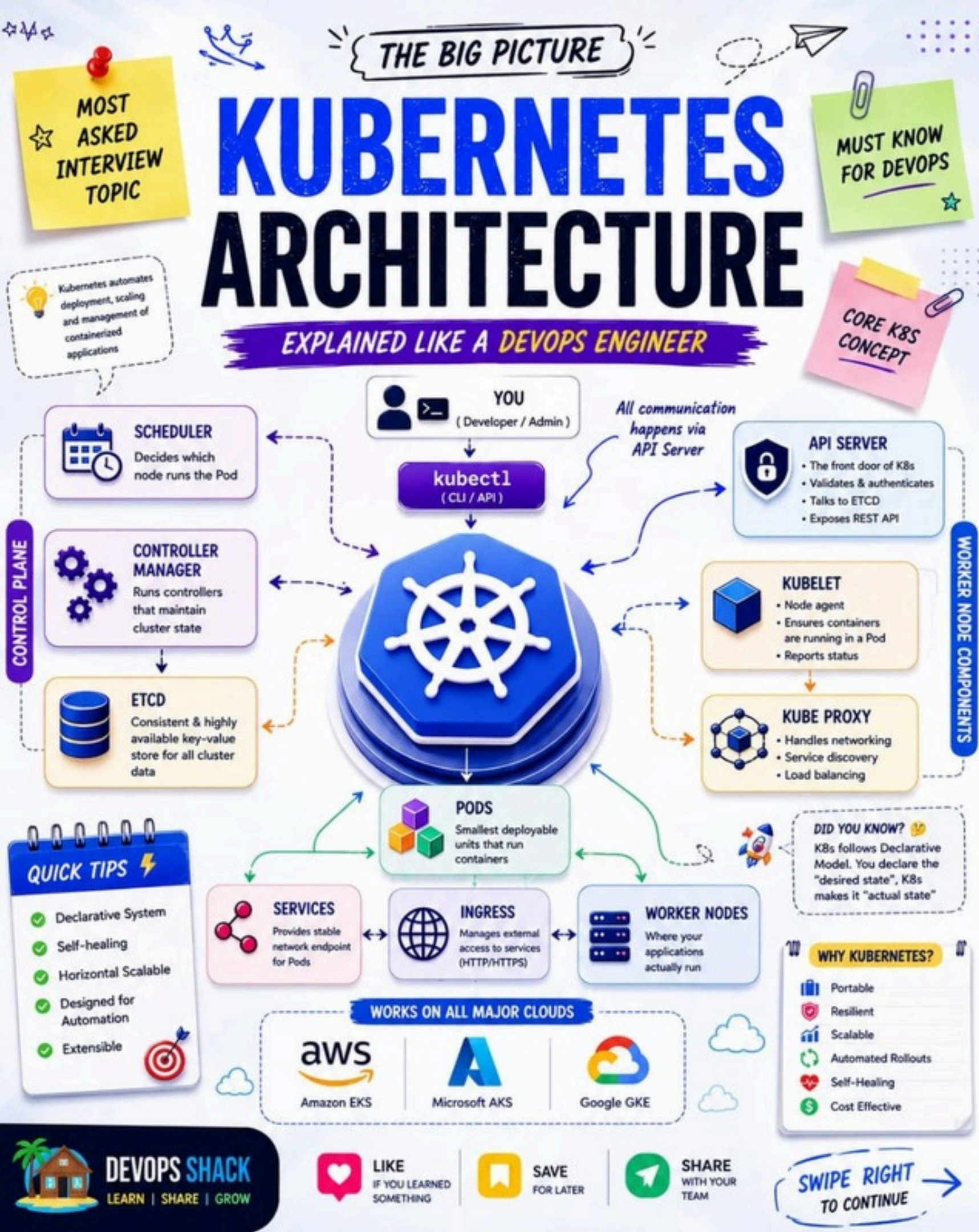


Utilisateur Final

Workflow :

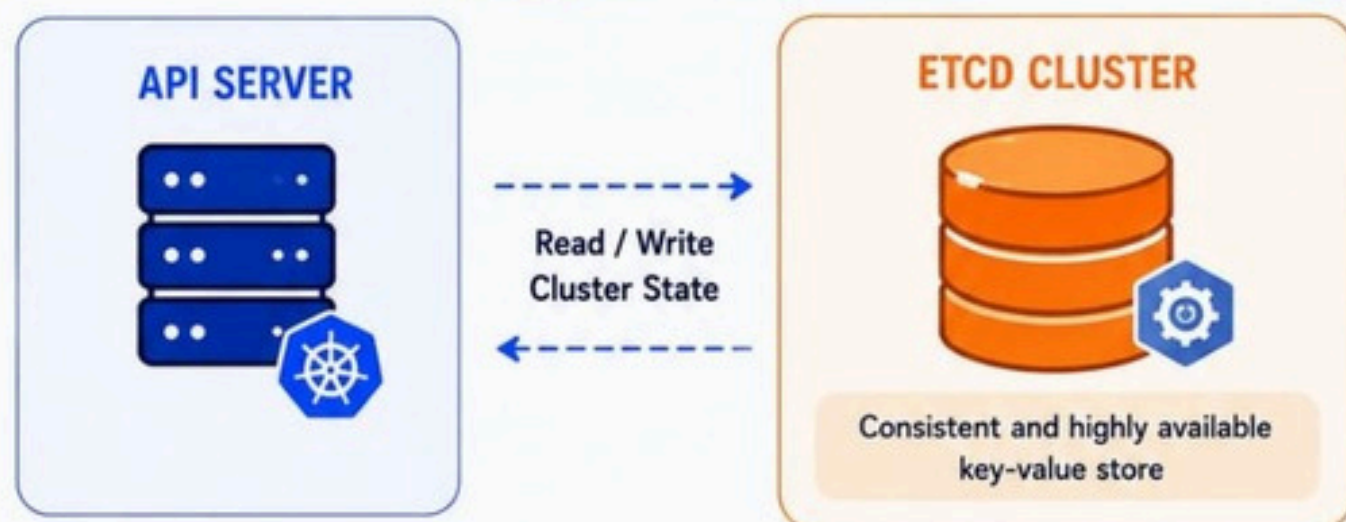
- Développement de l'application
- Création de l'image Docker
- Push vers un registre
- Déploiement via Helm
- Orchestration par Kubernetes
- Publication via Ingress
- Monitoring et Scaling automatique





ETCD

The *Heart* of Kubernetes



WHAT IT STORES



PODS

Information about all Pods



NODES

Information about all Nodes



SECRETS

All Secrets and credentials



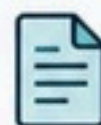
DEPLOYMENTS

Desired state of Deployments



SERVICES

ClusterIP, Endpoints, Service definitions



CONFIGMAPS

Configuration data

In short: Everything that defines the desired and current state of your cluster.

KEY CHARACTERISTICS



CONSISTENT

Strong consistency ensures cluster state is reliable



HIGHLY AVAILABLE

Runs in a cluster (odd number of nodes) to avoid downtime



DURABLE

Data is persisted on disk and survives failures



KEY-VALUE STORE

Simple, fast and optimized for storing cluster state



KEY TAKEAWAY

etcd is a distributed key-value database that stores all cluster data. Without etcd, Kubernetes has **no memory**.

CONTROL PLANE

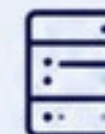
The *Brain* of Kubernetes

CONTROL PLANE COMPONENTS



API SERVER

The front door of Kubernetes. All communication goes through the API Server.



ETCD

Consistent and highly available key-value store for all cluster data.

Stores the state of the entire cluster.



SCHEDULER

Decides which node is best suitable to run a new Pod.

Looks at resources, constraints & policies.



CONTROLLER MANAGER

Runs controllers that maintain the desired state of the cluster.

Example: Node, ReplicaSet, Deployment, etc.

WHAT IT DOES



Validates & processes all API requests



Stores & retrieves cluster state



Schedules workloads efficiently



Ensures the actual state matches the desired state



Continuously monitors & reconciles

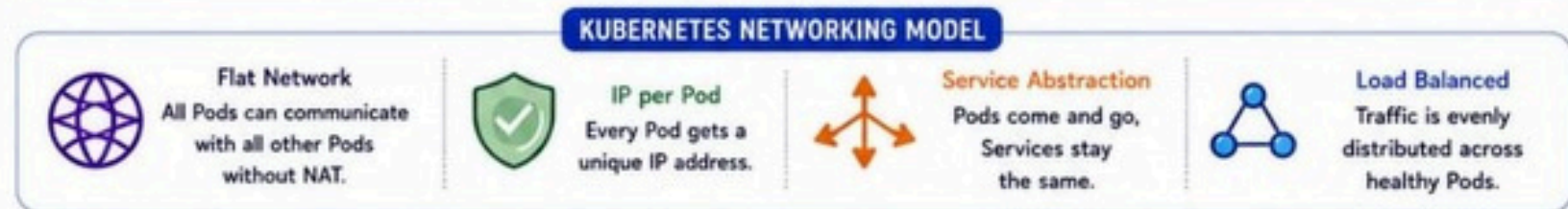
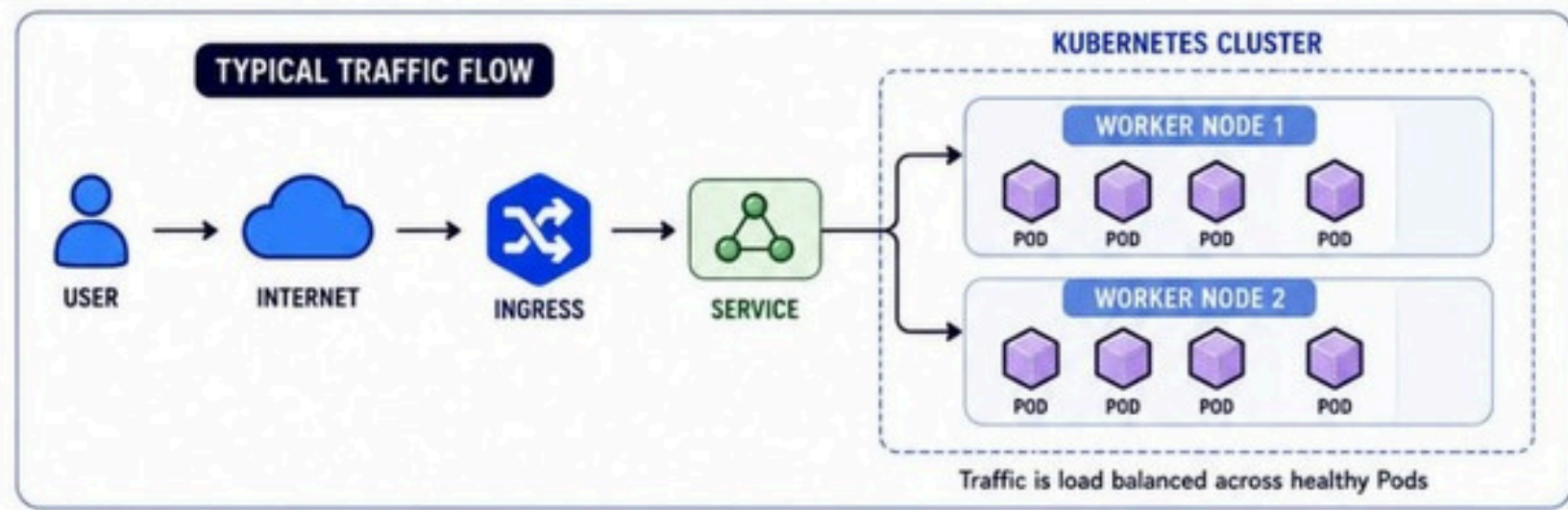
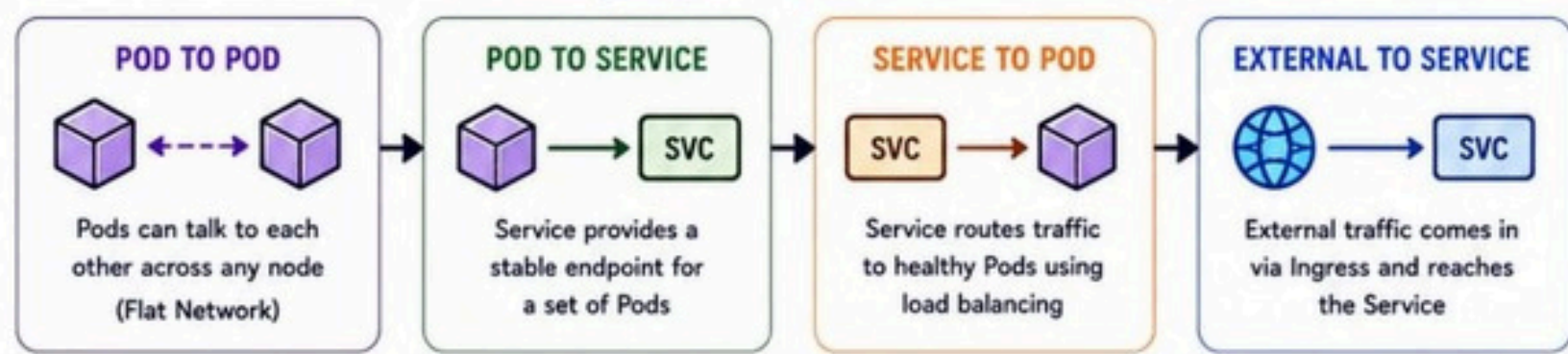


KEY TAKEAWAY: The Control Plane makes global decisions for the cluster. It does not run your applications.



KUBERNETES NETWORKING

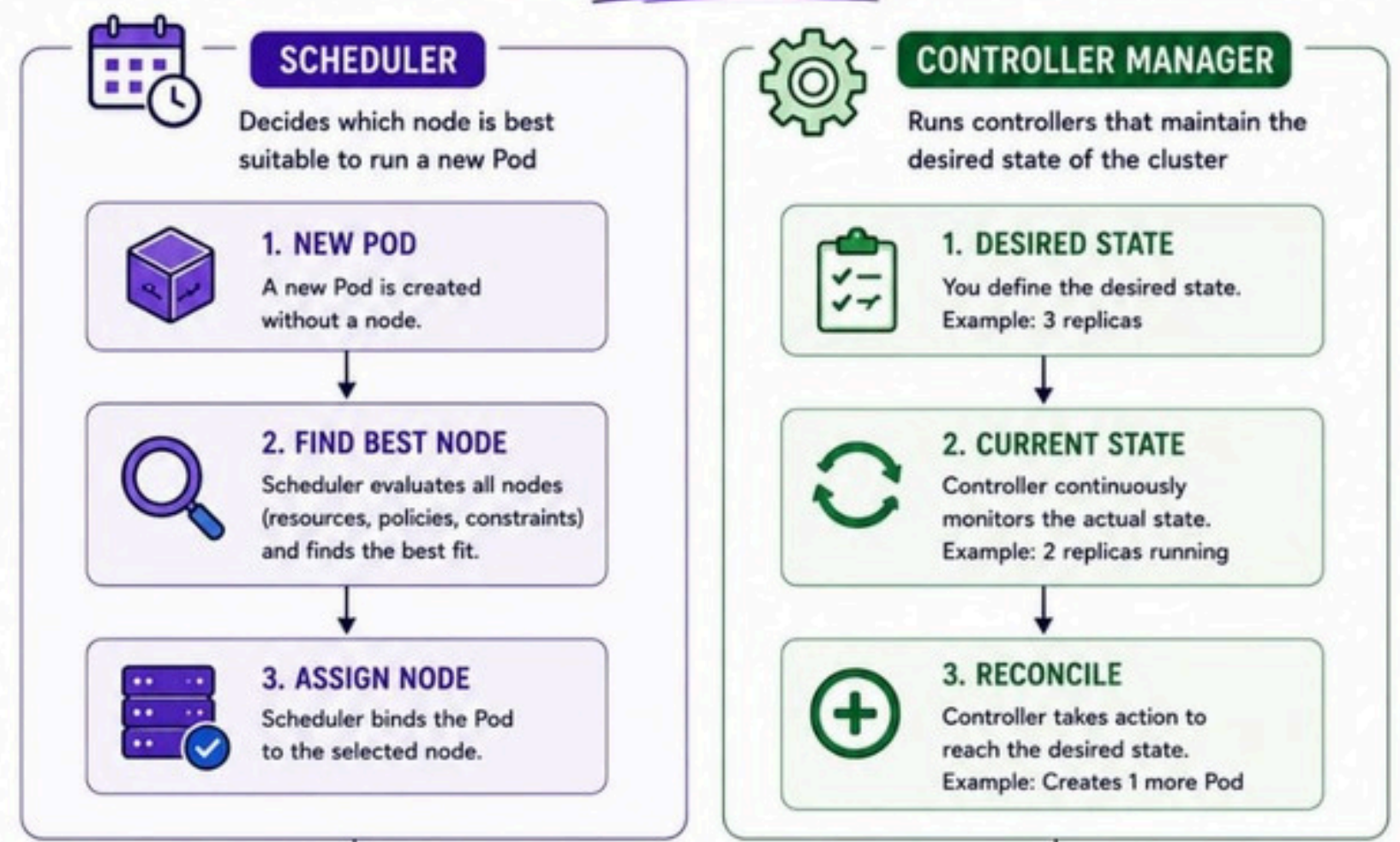
How Traffic Flows in a Cluster



KEY TAKEAWAY Kubernetes provides a flat, reliable and load-balanced networking model so your applications stay connected and highly available.

SCHEDULER & CONTROLLER MANAGER

The Decision Makers



KEY TAKEAWAY

- Scheduler makes placement decisions.
- Controller Manager makes sure the cluster stays in the desired state.

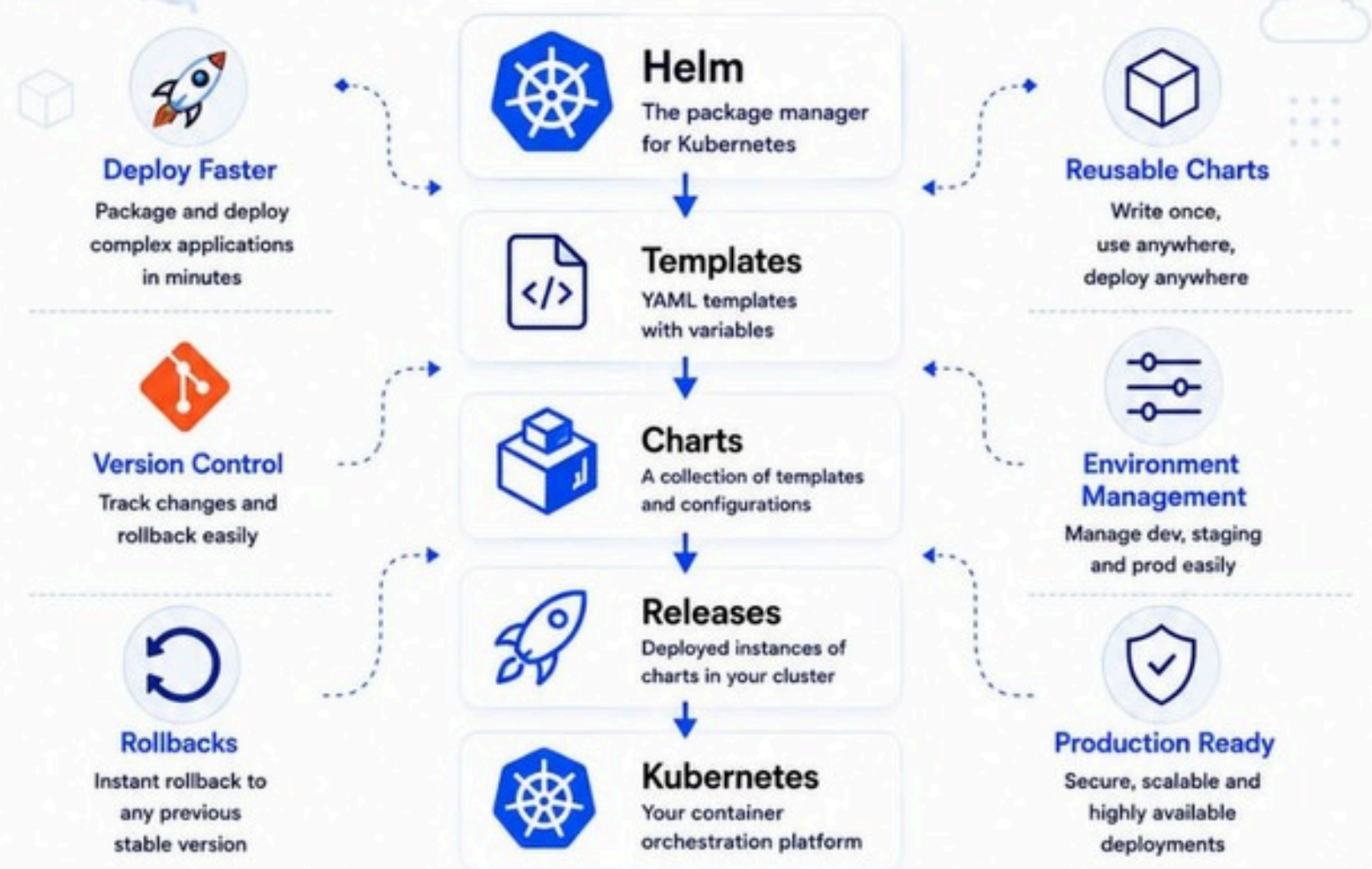
01/10

Helm Deployments

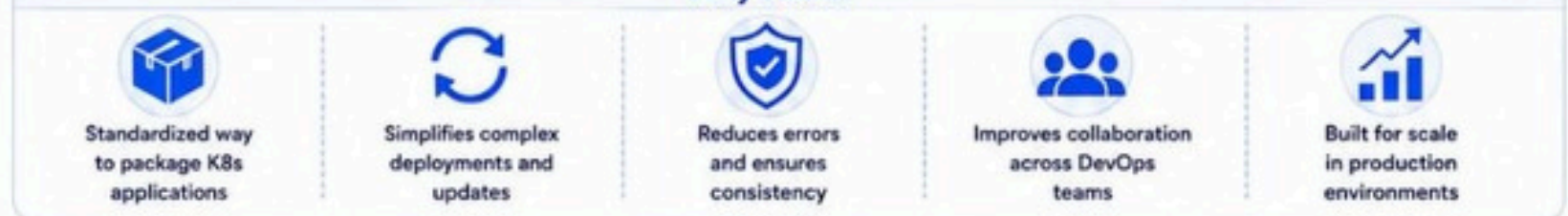
The Complete Production Guide



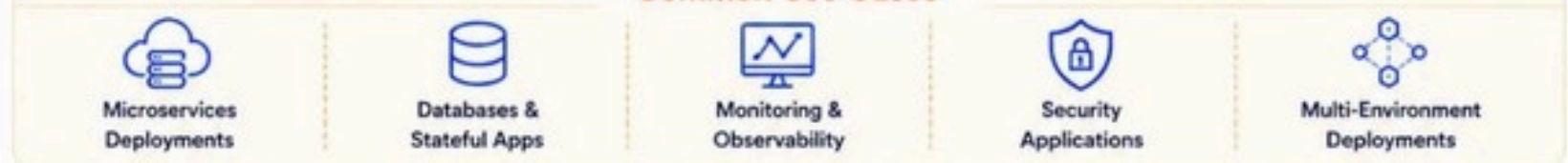
★ PACKAGE • DEPLOY • MANAGE • SCALE ★



Why Helm?



Common Use Cases

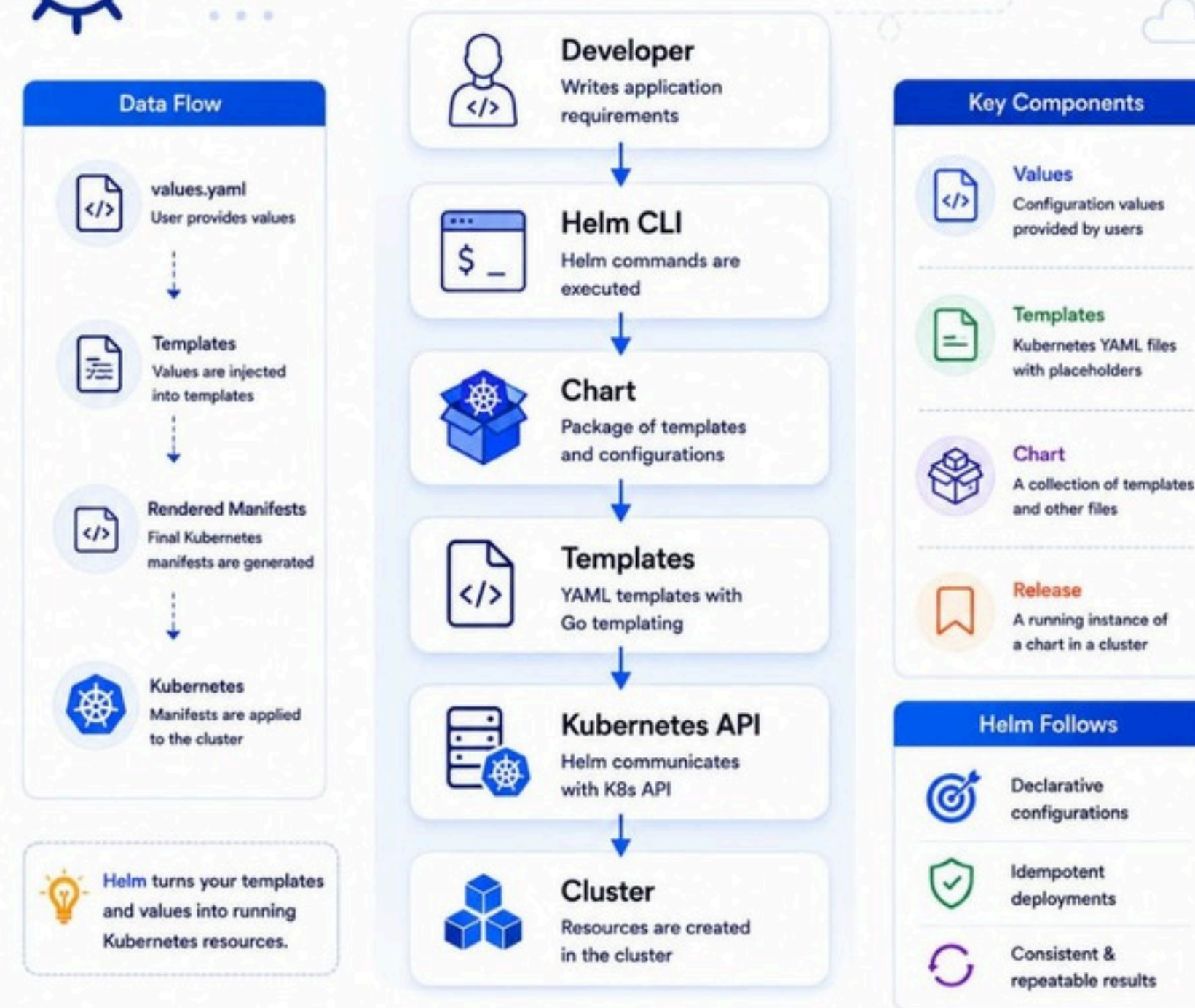


Helm helps you ship **reliable**, **repeatable** and **maintainable** Kubernetes applications.

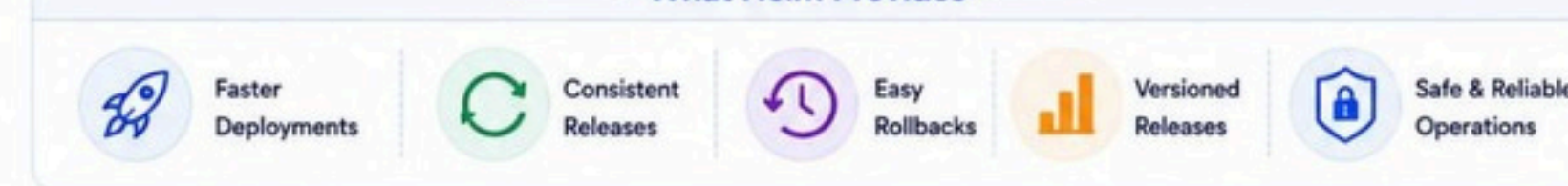
03/10

Helm Architecture

How Helm works behind the scenes



What Helm Provides



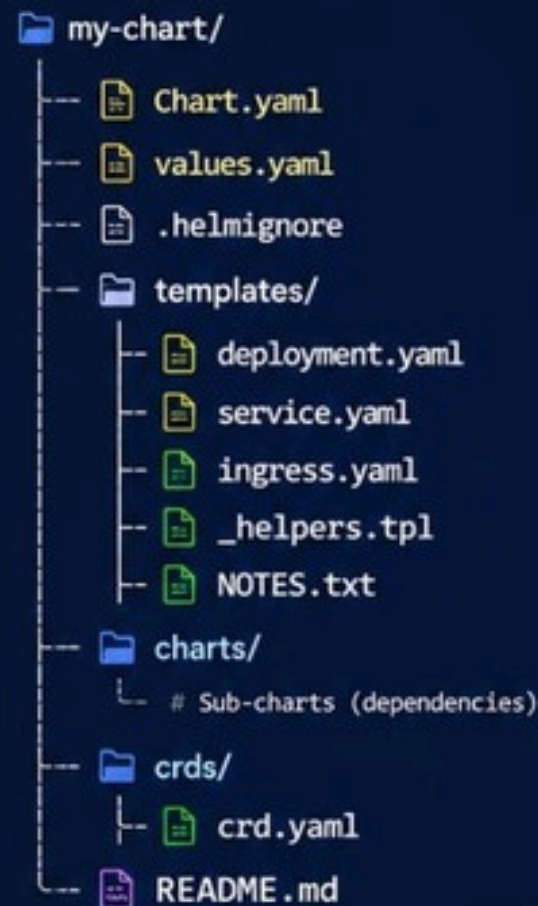


Helm Chart Structure

A Helm chart is a **package** that contains all the resources needed to deploy an application.



Example Chart



Minimal requirement:
Chart.yaml + templates/

File / Directory	Purpose
Chart.yaml	Metadata about the chart (name, version, description, appVersion, dependencies)
values.yaml	Default configuration values for this chart. Can be overridden by users.
.helmignore	Files and directories to ignore when packaging the chart.
templates/	Contains Kubernetes manifest templates. These are rendered with values.
*.yaml (inside templates/)	Kubernetes resource templates (deployment, service, ingress, etc.)
_helpers.tpl	Template helper functions that can be reused across templates.
NOTES.txt	Custom notes displayed after a successful installation.
charts/	Stores dependent charts (sub-charts). Used for modularity.
crds/	Contains Custom Resource Definitions (CRDs) to be installed before templates.
README.md	Information about the chart, usage, and configuration.

How It Works



Key Takeaways

- ✓ A Helm chart is a collection of files that describe a related set of Kubernetes resources.
- ✓ Templates + Values = Rendered Kubernetes Manifests.
- ✓ Charts make your deployments reusable, versionable and shareable.

Pro Tip

Keep your charts small, focused and reusable. Use values.yaml for all configurable parameters.



Helm Templates Explained

Helm uses **Go templates** to inject values and generate Kubernetes manifests **dynamically**.



1. values.yaml (Input)

```

image:
  repository: nginx
  tag: 1.25
service:
  type: ClusterIP
  port: 80
replicaCount: 2
env:
  - name: ENV
    value: production
  - name: LOG_LEVEL
    value: info
  
```

Users provide values in values.yaml (or custom file)

2. Template (Go Template)

```

templates/deployment.yaml

spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: app
          image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
          ports:
            - containerPort: {{ .Values.service.port }}
          env:
            {{- range .Values.env }}
            - name: {{ .name }}
              value: {{ .value | quote }}
            {{- end }}
  
```

Templates use {{ }} to access values and render the final manifest.

3. Rendered Manifest (Output)

```

deployment.yaml (rendered)

spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: app
          image: nginx:1.25
          ports:
            - containerPort: 80
          env:
            - name: ENV
              value: "production"
            - name: LOG_LEVEL
              value: "info"
  
```

Helm renders templates with values to produce valid Kubernetes YAML.

Common Template Syntax

{{ .Values.key }}	{{ .Values.image.repository }} → nginx	{{ range .Values.list }} ... {{ end }}	{{- range .Values.env }} - name: {{ .name }} value: {{ .value }} {{- end }}
{{ .Values.nested.key }}	{{ .Values.service.port }} → 80	{{ include "template" . }}	{{ include "mychart.labels" . }}
{{ default "val" .Values.key }}	{{ default "nginx" .Values.image.repository }}	{{ .Release.Name }}	{{ .Release.Name }} → my-app
{{ if .Values.key }} ... {{ end }}	{{ if .Values.ingress.enabled }} # ingress manifest {{ end }}		

Template Flow



What Happens?

1. User provides values
2. Helm reads templates
3. Go template engine renders YAML
4. Helm sends to K8s API
5. Resources are created

Pro Tip

Keep logic in templates simple & readable

Use values.yaml for all configurable data

Validate with helm template

Use helm lint to check template errors



Helm Release Lifecycle

Manage your applications across their entire lifecycle with simple Helm commands.



1 INSTALL

`helm install my-app ./my-chart`

Install a new chart as a release in the cluster.

What Happens?

- ✓ Helm renders templates
- ✓ Kubernetes resources are created
- ✓ A new release is recorded

2 UPGRADE

`helm upgrade my-app ./my-chart`

Upgrade an existing release with new changes.

What Happens?

- ✓ Helm calculates the diff
- ✓ Resources are updated in-place
- ✓ Release history is updated

3 HISTORY

`helm history my-app`

View the revision history of a release.

What Happens?

- ✓ Shows all revisions with status
- ✓ Displays chart, app version, time
- ✓ Helps in troubleshooting

4 ROLLBACK

`helm rollback my-app 2`

Roll back to a previous revision.

What Happens?

- ✓ Reverts to the specified revision
- ✓ Resources updated to the old state
- ✓ New revision is created

5 UNINSTALL

`helm uninstall my-app`

Remove a release and all its resources.

What Happens?

- ✓ Kubernetes resources are deleted
- ✓ Release record is removed
- ✓ Clean up the namespace (if used)

Real-World Use Cases

Install

Deploy a new application to the cluster

Upgrade

Roll out new features or configuration

History

Investigate issues or verify changes over time

Rollback

Quickly revert when something goes wrong

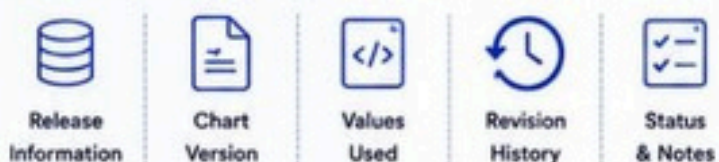
Uninstall

Remove applications cleanly

Release Revision Flow



Helm Stores Release Data



- Pro Tips**
- Always use meaningful release names.
 - Use values files for consistent deployments.
 - Check history before rollback.
 - Automate upgrades with CI/CD.



Helm Repositories

Helm repositories are collections of charts that you can share and install.



What is a Helm Repository?

A Helm repository is a location (HTTP server) where charts are stored in an `index.yaml` file.

Why Use Repositories?

- ✓ Share and reuse charts easily
- ✓ Discover and search charts
- ✓ Version management
- ✓ Consistent and reliable deployments

Popular Helm Repositories

Repository	URL	Description
Artifact Hub	artifacthub.io	The central hub for Kubernetes packages.
Bitnami	charts.bitnami.com	Helm charts for popular open source software.
GitHub Pages	username.github.io	Host your own charts using GitHub Pages.
Internal Repo	intranet.company.com	Private charts for your organization's applications.

How Helm Works with Repositories



Essential Helm Repository Commands

Command	Description	Example
<code>\$ helm repo add <name> <url></code>	Add a new repository	<code>\$ helm repo add bitnami https://charts.bitnami.com/bitnami</code>
<code>\$ helm repo list</code>	List all configured repositories	<code>\$ helm repo list</code>
<code>\$ helm repo update</code>	Update local index from all repos	<code>\$ helm repo update</code>
<code>\$ helm search repo <keyword></code>	Search charts in repositories	<code>\$ helm search repo nginx</code>
<code>\$ helm pull <chart> --repo <name></code>	Download a chart locally	<code>\$ helm pull nginx --repo bitnami</code>
<code>\$ helm repo remove <name></code>	Remove a repository	<code>\$ helm repo remove bitnami</code>

Repository Files

index.yaml (Mandatory)

Contains metadata about all charts in the repository.

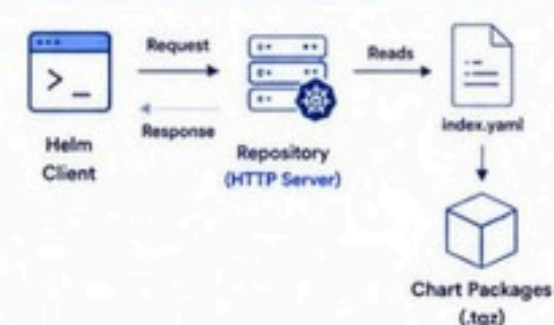
Chart Package (.tgz)

Compressed chart files stored in the repository.

Example: Using a Repository

- `$ helm repo add bitnami https://charts.bitnami.com/bitnami`
- `$ helm repo update`
- `$ helm search repo mysql`
- `$ helm install my-mysql bitnami/mysql`

Under the Hood



Pro Tip

You can host your own private Helm repository using:

- GitHub Pages
- NGINX
- S3 + Static Website
- Harbor OCI Registry (Helm OCI)

Keep your charts versioned and store them in Git!

Key Takeaway

Repositories help you discover, share, and manage Helm charts efficiently—whether public or private!

Remember

- ✓ Add → Update → Search → Install → Manage
- ✓ Keep your repositories updated for the latest charts.



GROUPE 3 | Meilleur Groupe Exposé



MERCI !

L'écosystème Cloud Native repose sur quatre briques majeures :

- Docker : création et exécution des conteneurs.
- Kubernetes : orchestration et gestion du cluster.
- NGINX Ingress Controller : exposition sécurisée des applications.
- Helm : industrialisation et automatisation des déploiements.

Cette combinaison constitue aujourd'hui le standard de facto pour les plateformes modernes DevOps, SRE et Cloud Native.